

IES NERVIÓN

Departamento de Informática

---

# PROYECTO FINAL

Sistema de Gestión de una Ludoteca

---

**Alumno:** Sergio González

**Ciclo:** 1º Desarrollo de Aplicaciones Multiplataforma (DAM)

**Módulo:** Programación

**Fecha:** Mayo 2026

**Temática:** Sistema de Gestión de una Ludoteca

Curso 2025–2026

# Índice

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1. Descripción del sistema</b>                         | <b>2</b>  |
| <b>2. Análisis y diseño</b>                               | <b>2</b>  |
| 2.1. Arquitectura de paquetes . . . . .                   | 2         |
| 2.2. Diagrama de clases UML . . . . .                     | 3         |
| 2.3. Justificación de las colecciones . . . . .           | 3         |
| <b>3. Manual de usuario</b>                               | <b>4</b>  |
| 3.1. Menú principal . . . . .                             | 4         |
| 3.2. Gestión de juegos . . . . .                          | 5         |
| 3.3. Gestión de socios . . . . .                          | 7         |
| 3.4. Gestión de préstamos . . . . .                       | 8         |
| 3.5. Gestión de torneos . . . . .                         | 10        |
| <b>4. Explicación técnica de los conceptos aplicados</b>  | <b>12</b> |
| 4.1. Herencia y polimorfismo . . . . .                    | 12        |
| 4.2. Clases abstractas e interfaces . . . . .             | 13        |
| 4.3. Colecciones y genéricos . . . . .                    | 13        |
| 4.4. Gestión de excepciones . . . . .                     | 14        |
| 4.5. Persistencia en ficheros . . . . .                   | 14        |
| 4.6. Streams y expresiones lambda . . . . .               | 15        |
| 4.7. Interfaz de usuario por consola . . . . .            | 15        |
| <b>5. Dificultades encontradas y soluciones adoptadas</b> | <b>16</b> |
| <b>6. Conclusiones</b>                                    | <b>16</b> |
| <b>7. Referencias</b>                                     | <b>17</b> |

# 1. Descripción del sistema

El sistema desarrollado es una aplicación de consola para gestionar una ludoteca, es decir, un local donde los socios pueden consultar el catálogo de juegos de mesa, realizar préstamos a domicilio, participar en torneos y dejar valoraciones sobre los juegos.

La aplicación está pensada para ser usada por el personal del local desde un ordenador de escritorio. No está orientada al socio directamente, sino al empleado que gestiona el día a día del negocio.

Las funcionalidades principales son las siguientes:

- Gestión del catálogo de juegos de cuatro tipos: tablero, cartas, dados y rol.
- Registro y gestión completa de socios.
- Control de préstamos y devoluciones con detección automática de vencimientos.
- Organización de torneos con inscripción de socios y control de plazas.
- Valoración de juegos por parte de los socios.
- Persistencia automática de todos los datos entre ejecuciones mediante ficheros CSV.

# 2. Análisis y diseño

## 2.1. Arquitectura de paquetes

El proyecto se organiza en cinco paquetes con responsabilidades bien definidas, siguiendo una arquitectura por capas:

- **modelo**: contiene todas las entidades del dominio. Es la capa más pura del sistema, sin dependencias hacia otras capas. Incluye las clases de juegos, personas, préstamos, torneos, reseñas y la clase genérica `ResultadoSistema<T>`.
- **repositorio**: gestiona el almacenamiento en memoria de cada entidad mediante colecciones Java. También incluye `GestorCSV`, responsable de la carga y guardado de datos en ficheros.
- **servicio**: contiene la lógica de negocio. Actúa como intermediario entre la interfaz de usuario y los repositorios, y es donde se aplican los Streams y expresiones lambda.
- **ui**: contiene los menús de consola. Solo se comunica con la capa de servicio, nunca directamente con los repositorios.

- **excepciones:** agrupa las tres excepciones personalizadas del dominio.

## 2.2. Diagrama de clases UML

A continuación se detalla la estructura lógica del sistema mediante el diagrama de clases UML, donde se especifican las relaciones de herencia de los modelos, las interfaces del negocio y la comunicación entre los paquetes de la aplicación.

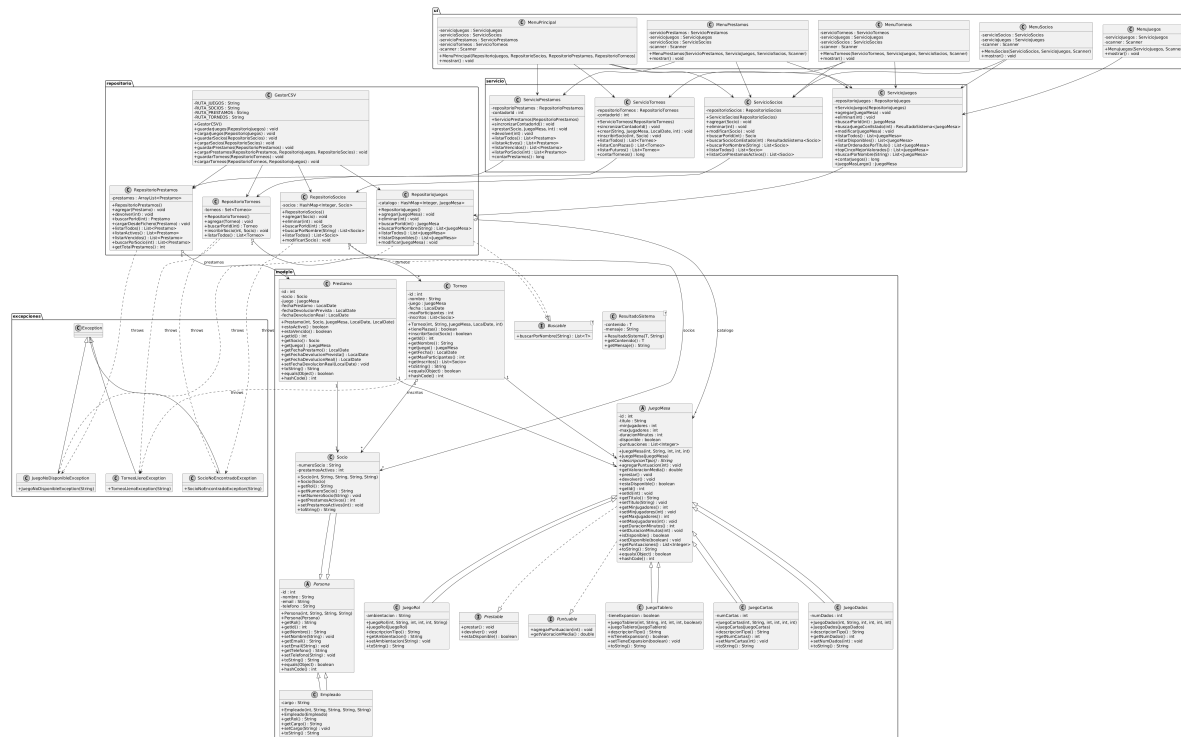


Figura 1: Diagrama de clases UML del sistema de gestión de la ludoteca.

## 2.3. Justificación de las colecciones

| Colección                   | Usada en             | Justificación                           |
|-----------------------------|----------------------|-----------------------------------------|
| HashMap<Integer, JuegoMesa> | RepositorioJuegos    | Acceso directo por id en O(1)           |
| HashMap<Integer, Socio>     | RepositorioSocios    | Acceso directo por id en O(1)           |
| ArrayList<Prestamo>         | RepositorioPrestamos | Orden de registro, recorrido secuencial |
| TreeSet<Torneo>             | RepositorioTorneos   | Ordenación automática por fecha         |

Cuadro 1: Colecciones usadas y su justificación

Se eligió `HashMap` para los repositorios de juegos y socios porque la operación más frecuente es la búsqueda por identificador, y el `HashMap` la realiza en tiempo constante sin necesidad de recorrer todos los elementos. Para los préstamos se eligió `ArrayList` porque se accede a ellos en orden de registro y se recorren de forma secuencial. Para los torneos se eligió `TreeSet` porque mantiene los torneos ordenados por fecha de celebración de forma automática.

### 3. Manual de usuario

Al ejecutar la aplicación, el sistema carga automáticamente los datos desde los ficheros CSV y muestra el menú principal.

#### 3.1. Menú principal

El menú principal permite navegar a las cuatro secciones de la aplicación. Al elegir la opción 0, el sistema guarda todos los datos automáticamente antes de cerrar.

```
Cargando base de datos de la ludoteca...
Datos cargados correctamente.

=====
          GESTIÓN DE LA LUDOTECA
=====
1. Gestión de juegos
2. Gestión de socios
3. Gestión de préstamos
4. Gestión de torneos
0. Salir
=====
Elige una opción: |
```

Figura 2: Menú principal de la aplicación

### 3.2. Gestión de juegos

Permite añadir juegos de cuatro tipos distintos, cada uno con sus atributos específicos. También permite eliminar, modificar, buscar por id o nombre, listar todos los juegos o solo los disponibles, ver el top 5 mejor valorado y el juego con mayor duración estimada.

```
-----  
Elige una opción: 1  
|  
--- GESTIÓN DE JUEGOS ---  
1. Añadir juego  
2. Eliminar juego  
3. Modificar juego  
4. Buscar juego por id  
5. Buscar juego por nombre  
6. Listar todos los juegos  
7. Listar juegos disponibles  
8. Top 5 mejor valorados  
9. Juego más largo  
0. Volver  
Elige una opción:
```

Figura 3: Menú de gestión de juegos

```
--- GESTIÓN DE JUEGOS ---
1. Añadir juego
2. Eliminar juego
3. Modificar juego
4. Buscar juego por id
5. Buscar juego por nombre
6. Listar todos los juegos
7. Listar juegos disponibles
8. Top 5 mejor valorados
9. Juego más largo
0. Volver
Elige una opción: 1

-- Añadir juego --
Id: 1
Título: Monopoly
Mínimo de jugadores: 2
Máximo de jugadores: 6
Duración estimada (minutos): 45
Tipo: 1-Tablero 2-Cartas 3-Dados 4-Rol
Elige tipo: 1
¿Tiene expansión? (s/n): n
Juego añadido correctamente.
```

Figura 4: Proceso de añadir un nuevo juego

### 3.3. Gestión de socios

Permite dar de alta nuevos socios introduciendo nombre, email, teléfono y número de socio. También permite eliminar, modificar datos, buscar por id o nombre, listar todos los socios o solo los que tienen préstamos activos.

```
Elige una opción: 2
|
=====
                GESTIÓN DE SOCIOS
=====
1. Registrar nuevo socio
2. Dar de baja socio
3. Modificar datos de socio
4. Buscar socio por ID
5. Buscar socio por nombre
6. Listar todos los socios
7. Valorar un juego
0. Volver
=====
Elige una opción:
```

Figura 5: Menú de gestión de socios



```
Elige una opción: 1

-- Registrar Nuevo Socio --
Introduce el ID que deseas asignarle (ej: 24): 1
Nombre completo: Sergio González
Email: adminludoteca@admin.com
Teléfono: 615890342
Número de socio: 1
Socio registrado correctamente con el ID: 1
```

Figura 6: Proceso de registrar un nuevo socio

### 3.4. Gestión de préstamos

Permite realizar un préstamo indicando el id del socio, el id del juego y el número de días. El sistema comprueba automáticamente que el juego esté disponible antes de registrarlo. También permite registrar devoluciones y listar todos los préstamos, los activos o los vencidos.

```
Elige una opción: 3

--- GESTIÓN DE PRÉSTAMOS ---
1. Realizar préstamo
2. Registrar devolución
3. Listar todos los préstamos
4. Listar préstamos activos
5. Listar préstamos vencidos
6. Listar préstamos de un socio
0. Volver
Elige una opción:
```

Figura 7: Menú de gestión de préstamos

```
Elige una opción: 1

-- Realizar préstamo --
Id del socio: 1
Id del juego: 1
Días de préstamo: 5
Préstamo realizado correctamente.
```

Figura 8: Proceso de realizar un préstamo

### 3.5. Gestión de torneos

Permite crear torneos indicando nombre, juego, fecha y número máximo de participantes. También permite inscribir socios, listar todos los torneos, los que tienen plazas disponibles o los que se celebran en el futuro.

```
Elige una opción: 4

--- GESTIÓN DE TORNEOS ---
1. Crear torneo
2. Inscribir socio en torneo
3. Listar todos los torneos
4. Listar torneos con plazas
5. Listar torneos futuros
0. Volver
Elige una opción:
```

Figura 9: Menú de gestión de torneos

```
Elige una opción: 1

-- Crear torneo --
Nombre del torneo: Cush Cup Inicial
Id del juego: 1
Fecha (yyyy-MM-dd): 2026-05-28
Máximo de participantes: 8
Torneo creado correctamente.
```

Figura 10: Proceso de crear un torneo

Todas estas capturas son algunas de las funcionalidades que tiene el programa, te invito a que pruebes cada funcionalidad tú mismo.

## 4. Explicación técnica de los conceptos aplicados

### 4.1. Herencia y polimorfismo

Se han definido dos jerarquías de herencia con sentido semántico real, no forzado:

- JuegoMesa (abstracta) → JuegoTablero, JuegoCartas, JuegoDados, JuegoRol
- Persona (abstracta) → Socio, Empleado

El polimorfismo se aplica en los repositorios, donde las colecciones almacenan referencias de tipo `JuegoMesa` que en tiempo de ejecución apuntan a objetos de subclases concretas. Esto permite, por ejemplo, que `RepositorioJuegos` gestione cualquier tipo de juego sin conocer su tipo exacto:

```
HashMap<Integer, JuegoMesa> catalogo = new HashMap<>();
catalogo.put(1, new JuegoTablero(1, "Catan", 3, 6, 90, true));
catalogo.put(2, new JuegoCartas(2, "Uno", 2, 10, 30, 108));
```

Todos los métodos sobrescritos llevan la anotación `@Override`.

## 4.2. Clases abstractas e interfaces

`JuegoMesa` es abstracta porque no tiene sentido instanciar un juego genérico sin tipo concreto. Define el método abstracto `descripcionTipo()` que cada subclase implementa de forma distinta, e implementa las interfaces `Puntuable` y `Prestable` con métodos concretos.

Se han definido tres interfaces propias:

- **Prestable**: define el contrato de los juegos que pueden prestarse (`prestar`, `devolver`, `estaDisponible`). Al implementarla en `JuegoMesa`, todas las subclases heredan este comportamiento y se puede usar `Prestable` como tipo de referencia.
- **Puntuable**: define el contrato para entidades que pueden recibir valoraciones (`agregarPuntuacion`, `getValoracionMedia`). La implementa `JuegoMesa`.
- **Buscable<T>**: interfaz genérica que define `buscarPorNombre`. La implementan `RepositorioJuegos` y `RepositorioSocios`, permitiendo usarla como tipo en parámetros de métodos.

## 4.3. Colecciones y genéricos

Se usan tres tipos distintos de colecciones justificadas en la sección anterior. Además se implementa `ResultadoSistema<T>`, una clase genérica propia que encapsula el resultado de una operación junto con un mensaje de estado:

```
public class ResultadoSistema<T> {  
    private T contenido;  
    private String mensaje;  
  
    public ResultadoSistema(T contenido, String mensaje) {  
        this.contenido = contenido;  
        this.mensaje = mensaje;  
    }  
}
```

Se usa en los servicios para devolver resultados sin propagar excepciones hasta la interfaz de usuario, por ejemplo en `buscarJuegoConEstado` o `buscarSocioConEstado`.

## 4.4. Gestión de excepciones

Se han creado tres excepciones personalizadas propias de la lógica del dominio:

- **JuegoNoDisponibleException:** se lanza al intentar prestar un juego que ya está prestado o al buscar un juego inexistente.
- **SocioNoEncontradoException:** se lanza al buscar un socio que no está registrado.
- **TorneoLlenoException:** se lanza al intentar inscribir a un socio en un torneo sin plazas.

Todas se usan con bloques `try-catch` con mensajes claros para el usuario. En `GestorCSV` se usa `try-catch-finally` para garantizar el cierre del `BufferedReader` aunque ocurra un error:

```
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader(RUTA_JUEGOS));
    % leer 1 neas...
} catch (IOException e) {
    System.out.println("Error al cargar juegos: " + e.getMessage());
} finally {
    if (br != null) {
        try { br.close(); }
        catch (IOException e) {
            System.out.println("Error al cerrar fichero: " + e.getMessage());
        }
    }
}
```

## 4.5. Persistencia en ficheros

Al arrancar, `Main` llama a `GestorCSV` que carga los cuatro ficheros CSV respetando el orden de integridad referencial: primero juegos, luego socios y por último préstamos y torneos, que dependen de los anteriores. Al cerrar, el mismo `GestorCSV` guarda todos los datos.

Los ficheros son legibles por un humano, con un registro por línea y campos separados por comas:

```
1,Catan,3,6,90,true,TABLERO,true
2,Uno,2,10,30,true,CARTAS,108
```

Se usa `BufferedReader` y `BufferedWriter` para la lectura y escritura, y se gestiona el caso en que el fichero no exista en el primer arranque.

## 4.6. Streams y expresiones lambda

Se usan más de cinco operaciones distintas de la Stream API con sentido real, todas en la capa de servicios:

```
// filter: juegos disponibles
repositorioJuegos.listarTodos().stream()
    .filter(j -> j.isDisponible())
    .collect(Collectors.toList());

// sorted + limit: top 5 mejor valorados
repositorioJuegos.listarTodos().stream()
    .filter(j -> j.getValoracionMedia() > 0)
    .sorted((j1, j2) -> Double.compare(
        j2.getValoracionMedia(), j1.getValoracionMedia()))
    .limit(5)
    .collect(Collectors.toList());

// max + mapToInt: juego con mayor duraci n
repositorioJuegos.listarTodos().stream()
    .max(Comparator.comparingInt(j -> j.getDuracionMinutos()))
    .orElse(null);

// count: total de socios
repositorioSocios.listarTodos().stream().count();

// filter + sorted: torneos futuros ordenados
repositorioTorneos.listarTodos().stream()
    .filter(t -> !t.getFecha().isBefore(LocalDate.now()))
    .sorted(Comparator.comparing(t -> t.getFecha()))
    .collect(Collectors.toList());
```

## 4.7. Interfaz de usuario por consola

La aplicación tiene un menú principal con cuatro submenús. Cada submenú implementa el CRUD completo de su entidad. Se validan todas las entradas del usuario: entradas no numéricas, cadenas vacías y formatos de fecha incorrectos. El programa nunca termina abruptamente ante una entrada inválida.



## 5. Dificultades encontradas y soluciones adoptadas

1. **Persistencia de préstamos en CSV.** Al cargar préstamos desde fichero era necesario reconstruir las referencias a objetos `Socio` y `JuegoMesa` ya cargados. Se resolvió pasando los repositorios de juegos y socios como parámetros al método `cargarPrestamos` del `GestorCSV`.
2. **Ordenación del `TreeSet` de torneos.** `TreeSet` requiere que los elementos sean comparables o que se proporcione un `Comparator`. Se resolvió pasando un `Comparator` personalizado al constructor que ordena primero por fecha y en caso de empate por id, evitando que el `TreeSet` descarte torneos con la misma fecha.
3. **Sincronización de contadores de id tras carga de CSV.** Al cargar datos desde fichero los servicios no conocían el último id usado, lo que podría causar duplicados al crear nuevos registros. Se resolvió añadiendo el método `sincronizarContadorId` en `ServicioPrestamos` y `ServicioTorneos`, que calcula el máximo id existente usando un `Stream` con `mapToInt` y `max`.
4. **Propagación de excepciones hacia la interfaz de usuario.** Algunos métodos de los menús propagaban excepciones que no debían salir de esa capa. Se resolvió capturándolas con `try-catch` dentro de los propios métodos del menú y mostrando un mensaje claro al usuario.

## 6. Conclusiones

Este proyecto ha sido la primera vez que he integrado todos los conceptos del módulo en una aplicación real y completa. Lo más valioso ha sido entender cómo se relacionan las distintas capas del sistema y por qué cada decisión de diseño tiene consecuencias en el resto del código.

La parte que más tiempo llevó fue la persistencia CSV, especialmente la reconstrucción de objetos con referencias cruzadas al cargar los datos, como también los `Streams`. Como mejoras futuras consideraría añadir una interfaz gráfica con `JavaFX` para hacer la aplicación más cómoda de usar, y una base de datos relacional con `JDBC` para una persistencia más robusta que los ficheros CSV, que puede tener problemas si los datos contienen comas.

En general el proyecto me ha enseñado que el diseño previo es fundamental: las veces que tuve que cambiar algo a mitad del desarrollo fue siempre porque no había pensado bien la arquitectura desde el principio.

## 7. Referencias

- Apuntes del módulo de Programación — IES Nervión, curso 2025–2026.